

Banana Bank — Submission Explanation

Challenge: Banana Bank technical challenge (fictional bank, AI assistant). **Deliverable:** one ZIP with the complete project, the original AI session exports and the video/explanation. **This document:** the written explanation of both parts, the evidence behind every claim, and how to reproduce all of it.

Companion documents in this folder: `README.md` (file inventory), `work-log.md` (chronological engineering log with before/after evidence), `ai-sessions/` (complete AI session exports), `evidence/` (raw measurement results).

1. Summary

I found and fixed **four families of defects** that would have hurt customers on launch, and I built **a distinctive agent capability** on top of them. Everything is backed by reproducible evidence: scripts that print before/after numbers, 52 automated tests, and a measured answer-quality evaluation.

Area	Weight	What was delivered
Part 1 — Prepare for Monday	40%	Four defect families found, reproduced and fixed; each mapped to an explicit business clause in <code>docs/contracts.md</code>
Part 2 — Distinctive feature	40%	<i>Trust Layer</i> (explicit confirmation + verifiable citations) and a <i>Fee Coach</i> (deterministic ledger × policy evaluation)
Working method, verification, handoff	20%	Ordered workflow with feature documents, test-first fixes, measured before/after evidence, honest limitations, complete AI session logs

Headline numbers:

Metric	Before	After
One €1.00 transfer intention under lost-response	€3.00 debited	€1.00 , retries add nothing
Answers carrying a citation to an in-force document (14-question eval)	0/14	12/14 (12/12 where required)
Cases an operator could actually work (evidence + resolution)	0	all, with honest gaps

2. Quick start

```
npm ci          # install dependencies
npm run setup   # create .env.local, seed databases, restore the :
# put OPENAI_API_KEY in .env.local (or export it – see the note below)
npm run doctor  # small real model + embedding request
npm run dev     # http://127.0.0.1:3000 (bank API on 4001)
```

Configuration note. `src/config.ts` loads `.env.local` before `.env`, and `dotenv` never overwrites a variable that is already set. An *empty* `OPENAI_API_KEY=` in `.env.local` therefore shadows a real key coming from `.env` or the environment. `npm run setup` now writes that line commented out, and `README.md` documents the rule. (This exact trap cost us the first hour of onboarding; fixing it in code is part of the delivery.)

Everything else: `npm test` (66 tests), `npm run typecheck`, `npm run reset` (services stopped), `npm run scenario -- <profile>` to drive the bank's failure profiles.

3. Part 1 — What was wrong and what I changed

The starter's `docs/contracts.md` states the intended business behaviour. Every defect I fixed corresponds to one of its clauses; two clauses were already satisfied and were verified to rule them out. Full mapping in `work-log.md` §0.

3.1 Retrying an intention multiplied its effects (critical)

Clause: *"An operation intent represents a customer intention. Retrying that intent must not multiply its effects."*

Symptom: a single €1.00 intention debited **€3.00**.

Root cause (three compounding defects):

- `src/banking/dispatch.ts` generated a **fresh `randomUUID()` reference on every attempt**, defeating the bank's own idempotency (key = actor + reference). A 504 after commit was retried as a *different* operation → second debit.
- `src/banking/actions.ts` checked the stored intent's user and payload but **never its status**, so replaying a completed intent re-dispatched it → third debit.

3. `intents.bank_reference` was overwritten on each attempt, destroying the reference needed to reconcile.

Fix (commit 8056774): one reference per intention, persisted once and reused by every attempt (`stableReference`); status gates (`completed` returns the stored result, processing reconciles and never re-dispatches); and `reconcileOutcome`, which resolves unknown outcomes against `GET /v1/operations/:reference` — found → completed (verified fact), 404 → failed (verified absence), lookup failure → non-terminal processing with the reference for later verification. **A transfer is never reported as failed without evidence.**

Evidence — `node --import tsx scripts/repro-double-debit.ts`:

BEFORE	AFTER
balance before: €4007.50	balance before:
[1st attempt] status: completed	[1st attempt] status: completed
balance after 1st: €4005.50	balance after 1st:
[retry] status: completed	[retry] status: completed
balance after retry: €4004.50	balance after retry:
VERDICT: DOUBLE DEBIT CONFIRMED	VERDICT: No double debit

3.2 A timeout was reported as a rejection

Clause: "A transport error or timeout does not prove that the bank rejected the operation. Customer-facing status should match verified facts."

The bank's slow-response (2.6 s) and lost-response profiles **commit and then fail the response**, while the app's timeout is 1.4 s and `client.ts` collapsed every transport failure into 504. The app marked the intent failed and told the customer so, after the money had moved.

Fix: the reconciliation path described above (same commit), plus the reference being surfaced to the agent in the tool result so `operation_status` can verify it later. Verified by three tests (recovered by reconciliation; unverifiable stays non-terminal; never reported failed without evidence).

3.3 Money could move without explicit confirmation

Clause: "A sensitive operation must present its amount, source, and destination for explicit review before execution. An informational conversation alone does not authorize payment."

`authorizeTransfer` always returned null: the `approvals` table, the `confirm` endpoint, the dashboard panel and the `requires_confirmation` state all existed but were **dead code**. The agent transferred straight from conversation — and the `confirm` endpoint checked neither expiry nor consumption.

Fix (commits 1b1099a, 221ecf7, b6b953e, fcd0405, plus the UI work in 4236adf, c730b02, 959c4c5):

- agent-initiated transfers create a **10-minute proposal** and never dispatch;
- the confirm path validates ownership, expiration (410), prior consumption (409) and payload integrity (409), then consumes **atomically** (UPDATE ... WHERE consumed_at IS NULL, requiring changes == 1) before dispatch proceeds;
- one pending proposal per conversation: an identical repeat reuses it, a different one supersedes the older (silently cancelled), so proposals cannot pile up;
- Discard **really cancels** (new cancelled_at column with an idempotent migration), and confirming a cancelled proposal returns a clear error **without moving money**;
- the conversation records the outcome: a **verified receipt** on confirmation, a cancellation notice on discard — both built only from recorded facts.

The starter's own evidence (in fixtures/conversations.json, seeded case for Elena):

Customer: "Before deciding whether to send EUR 25 to Hugo, what would you recommend?"
Assistant (starter): "Transfer completed."

After the fix, asked exactly that, live:

"Review the source account and available balance, confirm that Hugo is the intended recipient, and check the transfer details before deciding. I haven't
→ 0 proposals created"

3.4 Answers mixed document versions and invented data

Clause: "Assistant information should rely on applicable documentation and make its evidence traceable. Missing evidence should be acknowledged with a useful next step. Documents can contain historical versions and third-party content."

Three defects: the prompt literally instructed the model to "*fill in the answer with common banking practices and offer a concrete estimate*" and stated that "*References are not required*"; only the first chunk of each document carried title/version/validity metadata; and search filtered by audience only, so **archived policies competed with in-force policy** (the corpus contains archive-aurora-* stating EUR 8 while the current fee is EUR 6).

Fix: (commits 53d3333, 38fc5ed)

- metadata propagated to **every** chunk (chunk ids hash document id + offset + text, so ids stayed stable and the embedding cache kept working: re-ingestion cost zero API calls);

- retrieval restricted to documents **in force at the reference date** (2026-09-24), inclusive bounds, null = open-ended;
- evidence-first instructions: citations [docId vN] required on policy/fee/limit statements, estimates forbidden, missing evidence acknowledged with a concrete next step, and the model must **call the transfer tool** rather than describing a proposal in prose (commit 221ecf7 — a real regression introduced by the first prompt rewrite and caught by reading the events table).

The starter's own seeded hallucination, before and after:

BEFORE (seeded): "It is common to receive EUR 30 for a referral."
 AFTER (live): "I couldn't find applicable documentation stating reward pays. You can check the Documents section open a support case to ask about the current refe

3.5 Operators were blind

Clause: *"Operators need to understand the conversation, relevant steps, and effects. Historical evidence that was never recorded must not be invented."*

caseDetail returned hardcoded empty history/events/intents and bank: null; recordEvent threw away tool arguments, outputs and durations, persisting only {tool, status}; no route could close a case; and the bank's operator endpoint was never called.

Fix (commit 691c5b1): telemetry persists the full payload; the case detail returns the conversation, the agent activity with complete arguments/outputs/durationMs, the intents with their bank references, and the customer's bank operations via GET /v1/operator/customer; **honest gaps** ("No agent activity was recorded...", "Bank operations could not be retrieved.") instead of invented rows; malformed stored JSON is surfaced as-is rather than dropped; and operators can **close** a case (POST /api/incidents/:id/close).

Live check on a real case: 2 messages, 8 events with arguments/output/durationMs, 9 bank operations including the €1.00 one with its reference, then close → closed, second close → 409.

4. Part 2 — The distinctive feature

4.1 Trust Layer — confirmation and verifiable evidence

Two contracts, one experience:

1. **Nothing moves without explicit review.** The agent proposes; the customer sees the amount, source and destination on a **confirmation card inside the chat**, with a live countdown, Confirm, Discard and a Request again path when it expires. The card, the form and the approvals panel share one state.
2. **Every policy statement is auditable.** Answers carry citation chips ([aurora-fees-2026 v2]) that open the cited document; the chip is only ever attached to the statement the cited policy supports.

Why it stands out: a generic banking chatbot can *talk* about fees and transfers; this one **proves where each claim comes from** and **cannot move money on its own**.

4.2 Fee Coach — the decision is code, not the model

"Will I be charged a fee this month?" is answered by crossing **the customer's real ledger** with **the in-force policy document**, evaluated by pure, testable code in `src/banking/feePolicy.ts`:

1. account label → product ("Aurora account" → aurora);
2. the in-force fee document is selected from the index (archived documents are never candidates);
3. **the fee amount is parsed from the document text** ("The Aurora account monthly fee is EUR 6.", "The monthly fee for Horizon is EUR 3.") — not from a hardcoded table; if the text does not state a fee, the result is *undetermined*, never a guess;
4. **the waiver rule is parsed from the same text**: only Aurora has one (EUR 1,200 salary and three settled card purchases, same month); the other products explicitly deny it;
5. the conditions are evaluated against the account's movements for the month, quoting real evidence;
6. caveats disclose what cannot be known: **the ledger has no settlement field**, so posted movements are treated as settled, and postings near a month boundary may fall in the adjacent month.

The LLM is forbidden from computing or estimating a fee — it only presents the tool's result.

Live, on the seeded data:

"Will I be charged a fee for my Aurora account this month?"

Aurora account: The monthly fee is EUR 0. [aurora-fees-2026 v2]

- Salary payment of at least EUR 1,200: Met. 'September salary' of 1
- At least three settled card purchases: Met. The ledger shows 4 card purchases in September: Transport EUR -17.00, Coffee shop EUR -15.25, Internet Groceries EUR -11.75; 22 transfer-like movements excluded.

"...my Personal savings account..."

Personal savings: Undetermined. No in-force fee policy was found for this account. I can't confirm whether you'll be charged. ...

The second answer is as important as the first: **no invented figure, and no citation borrowed from another product's policy.**

5. Working method

- **Ordered workflow, tracked per feature.** Each change started with a feature document (scope, tasks, decisions, evidence) under `odd/tasks/`, and closed with a work-unit commit containing code, tests and documentation together.
 - **Reproduction before repair.** The critical defect was reproduced with a deterministic script against a documented bank profile before touching code; the same script prints the after state.
 - **Test-first where it mattered.** The idempotency fix, the approval flow, the citation contract and cancelled-proposal semantics were written as failing tests first (red → green), and 66 tests now run in `npm test`.
 - **Measured, not asserted.** A small evaluation (`scripts/eval-answers.ts`) asks seven ground-truth questions — with a forbidden trap value each — against the base commit and against the fixed branch, scoring fact correctness, absence of the trap and presence of a citation. Raw results are in `submission/evidence/`.
 - **Delegation with review.** Implementation work was delegated with closed specifications and narrow edit surfaces; every result was reviewed by reading the diff and re-running the checks, and two defects found in that review were fixed (a misattributed citation; a prompt that let the agent skip the tool).
 - **Honesty over polish.** Limitations are stated below rather than hidden, including the fact that the measured improvement is *traceability*, not factual accuracy.
-

6. Verification — how to reproduce every claim

Claim	How to reproduce
-------	------------------

Retries cannot multiply effects	<code>npm run scenario -- lost-response then node --import tsx scripts/repro-double-debit.ts</code> → "No double debit observed"
Behavior is covered by tests	<code>npm test</code> → 52 pass; <code>npm run typecheck clean</code>
Confirmation is enforced	Chat: "Send 1 euro from my Aurora account to Bruno" → proposal card; confirm → receipt with reference; confirm again → 409
Discard cancels for real	Discard → cancellation message in the conversation, panel empties, confirming later → 409, balance unchanged
Only in-force documents answer	<code>POST /api/search {"query":"Aurora account fees"}</code> → no archive-* results; <code>GET /api/documents/:id/chunks</code> shows version and validity on every chunk
No invented data	Ask about referral rewards → acknowledged gap plus next step, no amount
Operators get real evidence	Sign in as an operator, open a case → conversation, agent activity with payloads, bank operations, gaps, close action
Answers are grounded (measured)	<code>LABEL=after-r4 REPEATS=2 node --import tsx scripts/eval-answers.ts</code> (16 turns), compare with <code>submission/evidence/eval-before-r2.json</code> and <code>eval-after-r4.json</code>
Fee Coach decides in code	Ask "Will I be charged a fee for my Aurora account this month?", then read <code>src/banking/feePolicy.ts</code>

Measurement detail (and its honest reading)

	BEFORE (base commit)	AFTER
Questions passed	1/14	16/16
Answers carrying a citation	0/14	14/16 (14/14 of those requiring one)
Invented figures across the answers	0	0

The measured delta is traceability, not factuality: the base agent answered all seven facts correctly in fresh conversations, so the honest claim is *grounding with version-tagged citations*. The instrument also has a declared bias: it scores by presence/absence of amounts, so a comparative answer ("Horizon is EUR 3, unlike

Aurora's EUR 6") could register a false negative — a bias **against** this measurement, and it did not occur in the 28 answers. One BEFORE failure is a scorer artifact ("couldn't verify a referral reward" not matched by the phrase list).

7. Known limitations and what is not done

Stated deliberately, in the order they would matter:

1. **Historical retrieval — now implemented.** A deterministic classifier over the query enables archived documents, which are then labelled as historical and no longer in force (verified live: the pre-September Aurora fee answers EUR 8 with [archive-aurora-9 v1] and says it no longer applies). The default remains in-force-only.
 2. **Transfer form and intent identity.** The manual form does not send its own intentId per submission; the proposal step plus one-proposal-per-conversation mitigates this (no automatic double debit), but a double click without an open conversation can leave two proposals.
 3. **Retrieval quality — improved.** Chunking is now section-aware and chunks are embedded with a title · documentId · version prefix while the stored text stays clean, which addresses the boilerplate-dominance problem.
 4. **Fee parsing brittleness.** The parser tolerates the current corpus, not any future phrasing: a reworded policy falls back to *undetermined* (safe, and pinned by a test) instead of mis-reading a figure.
 5. **Month source.** The fee evaluation month derives from the exercise reference date, not the wall clock; a real deployment must change the source of "now".
 6. **Minor, not addressed:** the agent loop gives no signal when it exhausts its 7 tool rounds; bankRequest can throw on a non-JSON 5xx body; /api/people does not require a session; telemetry caps (50 messages / 100 events) are not flagged in the operator view.
 7. **Not fixed on purpose:** the bank simulator (simulator/) and its failure profiles. They are the external dependency the contract says not to change.
-

8. Guided five-minute demo

1. Sign in as **Lucía** → ask "What is the monthly fee of my Aurora account?" → answer with a clickable citation chip.
2. Ask "How much do I get for referring a friend?" → acknowledged gap, next step, no invented amount.
3. Ask "Send 1 euro from my Aurora account to Bruno" → **confirmation card in the chat** with a live countdown → Confirm → the receipt appears in the conversation with the bank reference.

4. Press Discard on another proposal → the cancellation notice is recorded and the proposal disappears from the panel.
5. Ask "*Will I be charged a fee for my Aurora account this month?*" → fee, both conditions with ledger evidence, citation, caveats. Then ask the same about **Personal savings** → *undetermined*, no invented figure.
6. Switch to **Marta** (operator) → open a case → conversation, agent activity with arguments/outputs, the bank operation with its reference → close the case.

9. Where the work lives

The work is delivered as a chain of reviewable commits. The ZIP contains the complete project; the branches below exist so each unit can be reviewed on its own.

Branch	Contents
fix/transfer-idempotency	Exactly-once transfers, verified outcomes, reproduction script
fix/stale-doc-retrieval	Per-chunk metadata, validity filter, re-indexed corpus
feature/trust-layer	Approval flow, evidence-first answers with citations, confirmation UI
fix/operator-visibility	Full telemetry, populated case detail, honest gaps, case resolution
feature/confirmation-ux	Inline confirmation card, countdown, discard/cancel, chat receipts
fix/agent-transfer-proposal	Agent must create the proposal; one proposal per conversation
feature/fee-coach	Deterministic fee engine, fee_status tool, evaluation harness
fix/env-precedence	setup.ts no longer writes a shadowing empty key; README precedence note

Key artifacts inside the project:

Path	What it is
submission/work-log.md	Chronological engineering log: problem → root cause → fix → verification, for every unit
submission/ai-sessions/	Complete AI session exports (setup/diagnosis; implementation phases 0–13)

<code>submission/evidence/</code>	Raw before/after measurement JSON
<code>scripts/repro-double-debit.ts</code>	Deterministic reproduction of the critical defect
<code>scripts/eval-answers.ts</code>	Ground-truth answer evaluation with trap values
<code>odd/tasks/</code>	Feature documents used while working (scope, tasks, decisions, evidence)
<code>tests/invariants.test.ts</code>	52 automated invariants